

set to 0. The fifth is the socket address structure, but unlike *sendto*, its purpose is to store the IP address and the port number of the sender. Thus, when *recvfrom* returns after receiving a datagram, you know 'who' has sent the datagram by checking the contents of that structure. Also, the number of bytes sent from that structure is returned in the sixth argument, *socklen_t *fromlen*, a pointer to this value.

Please note the two lines of code from the server source code, which are shown below:

```
numbytes = recvfrom(sd, datareceived, BUFSIZE, 0,
    (struct sockaddr*)&cliaddr, &length);
bytessent = sendto(sd, datareceived, numbytes, 0, (struct sockaddr
    *)&cliaddr, length);
```

The address structure *cliaddr* is populated in *recvfrom* and so after that function returns successfully, the structure contains the IP address and port number of the client, which is then used in the *sendto* function to direct the server's response to the requesting client. I hope the basic logic is now clear.

The *bind* function associates a local address with the socket. Normally, it is used on the server side. Please note the following fragment of code:

```
serveraddress.sin_family = AF_INET;
serveraddress.sin_port = htons(MYPORT); //PORT NO
serveraddress.sin_addr.s_addr = htonl(INADDR_ANY); //IP ADDRESS
ret = bind(sd, (struct sockaddr*)&serveraddress,
    sizeof(serveraddress));
```

The *bind* function associates the server's port number and IP address with the socket after it is populated. Here, *htonl(INADDR_ANY)* is used to bind to all IP addresses on all network interfaces; the *INADDR_ANY* constant is a 'wild card' address.

Also note the use of the *inet_ntop* function shown below:

```
printf("Data Received from %s",
    inet_ntop(AF_INET, &cliaddr.sin_addr,
        clientname, sizeof(clientname)));
```

This converts the 32-bit IPv4 address (which is in the network byte order) into a dotted decimal notation (presentation format), and stores that in the *clientname* array, whose size is specified by *sizeof(clientname)*.

Please note that the server code runs in an infinite loop; to terminate it, use *Ctrl-C*. (While writing production code, it's better to write a signal handler for *Ctrl-C* and do any clean-up that may be needed. I have omitted this for simplicity.)

Now let's try and analyse the running of the program, using the very useful packet analyser *tcpdump* (you need to be the *root* user to run it). I ran the server and client in one terminal each, and *tcpdump* in a third. The option *-i* is for specifying the capture interface; *-f* for printing without any time stamps, and

'*udp port 11710*' tells *tcpdump* to capture UDP packets with the port number 11710. Here is *tcpdump*'s captured output:

```
[root@localhost udp]# tcpdump -i lo -t 'udp port 11710'
tcpdump: verbose output suppressed, use -v or -vv for full
protocol decode
listening on lo, link-type EN10MB (Ethernet), capture size 65535
bytes
IP localhost.49485 > localhost.11710: UDP, length 6
IP localhost.11710 > localhost.49485: UDP, length 6
IP localhost.49485 > localhost.11710: UDP, length 5
IP localhost.11710 > localhost.49485: UDP, length 5
```

Here in two runs, from the client side the strings 'hello' and 'test' are sent to the server and echoed back. The first and third rows are data sent from the client to the server; the other two are the server's responses to the client. The string lengths are 6 and 5 since the newline character too is accepted by the *fgets* function used to get keyboard input in the client. The number 49485 is the port number of the client (and 11710, of course, of the server). For the client, the port number is chosen by the OS when *sendto* is called.

I hope from here on you will be able to extend your socket programming. In the next article, we take on the basics of socket programming in TCP. 

References

- [1] Stevens, W Richard, Rago and A Stephen, 'Advanced Programming in the Unix Environment', 2nd Edition, Pearson Education
- [2] Stevens W Richard, Fenner Bill and Rudoff A M, 'Unix Network Programming' (The Sockets Networking API – Vol. 1), Third Edition, Pearson Education

Useful Internet links

- [1] Internet Protocol Specification: <http://www.ietf.org/rfc/rfc791.txt>
- [2] User Datagram Protocol (UDP): <http://www.ietf.org/rfc/rfc768.txt>
- [3] For programming-related questions and answers: <http://www.askmybozhizone.com/>
- [4] <http://www.faqs.org/faqs/unix-faq/socket/>
- [5] <http://pubs.opengroup.org/onlinepubs/009604599/basedefs/netinet/in.h.html>
- [6] <http://en.wikipedia.org/wiki/Netstat>

Acknowledgement

I would like to thank Tanmoy Bandyopadhyay for his help in reviewing this article

By: Swati Mukhopadhyay

The author has more than 12 years of experience in academics and corporate training. Her interest areas are digital logic, computer architecture, computer networking, data structures, Linux, programming languages like C and C++, shell scripting and Python scripting. For any queries or feedback, she can be reached at swati.mukerjee@gmail.com.